CreditSense: Loan Risk Assessment Challenge

Best Public Solution Report

Johnny-Izhak-Kyriakos

Students: Johnny, Izhak, Kyriakos

Kaggle: JohnnyC0rp

Main submission path: `meta_overnight`

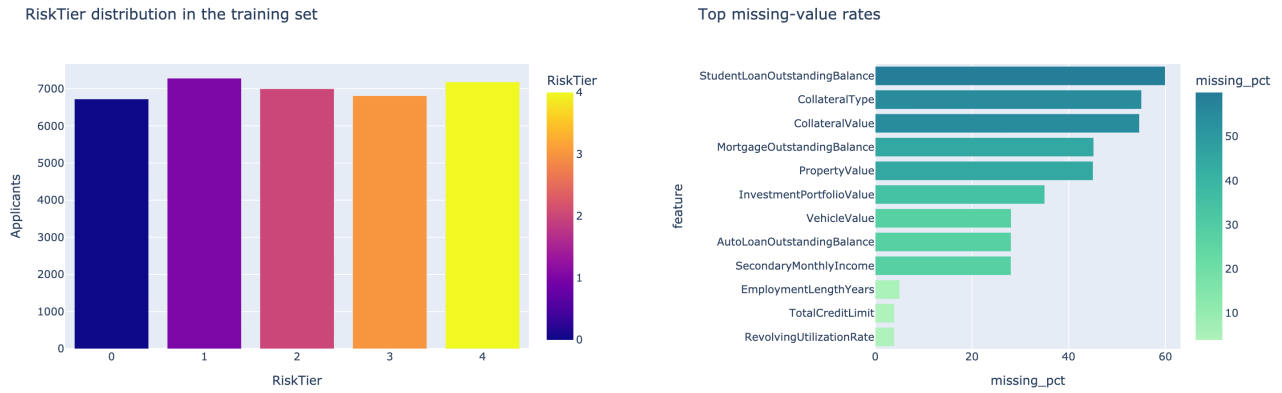
Public-best local validation: 0.8495 | Strongest later local score: 0.8576

This report focuses on the public submission lane we would actually hand in. The full project history became much wider than this final path, so here we keep the story tight: what the data looked like, which feature families stayed useful, and how the final ensemble was chosen. The longer methods timeline lives in the separate detailed report.

1 Data Exploration & Preprocessing

The competition asks for two outputs per customer: a five-class `RiskTier` and a continuous `InterestRate`. The public score is the simple average of classification accuracy and regression R^2, so we needed a pipeline that stayed balanced instead of over-optimizing only one target.

The first useful observation was that the data was not especially messy in the usual way. Class balance was already decent, so we did not need resampling tricks. The harder issue was missingness: many blanks were meaningful rather than accidental. Property fields often disappeared for renters, some debt fields were blank because the applicant simply did not carry that debt, and a few collateral fields behaved the same way. That pushed us toward explicit missingness flags instead of pretending that every blank cell meant the same thing.



(a) Risk tiers are reasonably balanced.

(b) Missingness is concentrated in meaningful financial columns.

Figure 1: The early data check was reassuring in one way and tricky in another: class balance was fine, but missing values clearly carried signal.

We kept preprocessing fairly plain on purpose:

- numeric columns were median-filled,

- categorical columns used a stable ordinal encoding,
- frequency features were added for each categorical value,
- one fixed stratified split was used for model comparison.

That last point mattered more than it sounds. Once the feature space became large, it was easy to fool ourselves with tiny validation swings. Keeping the split fixed made the branch comparisons much easier to trust.

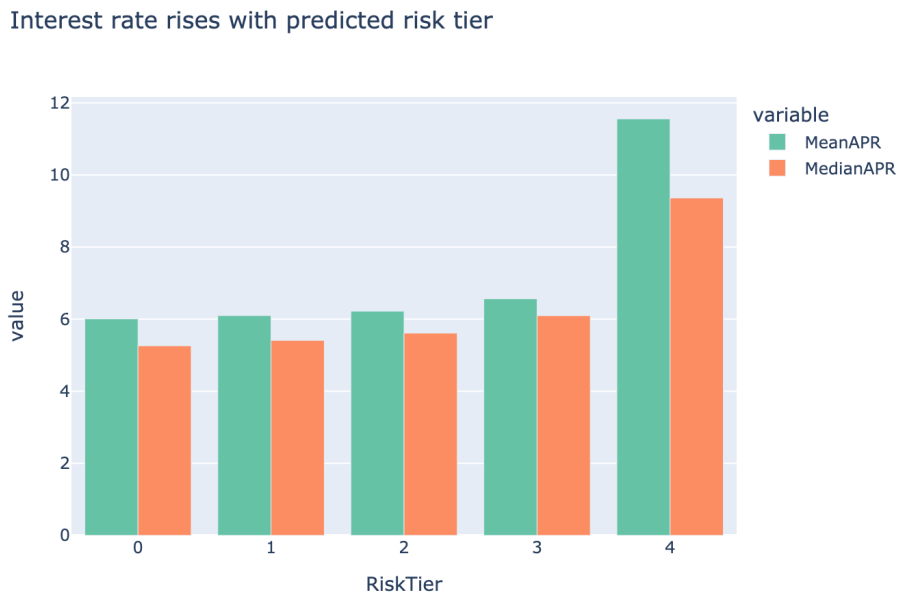


Figure 2: Higher risk tiers also carry higher interest rates, but the relationship is not perfectly discrete, which is why the regression target still needed its own attention.

2 Feature Engineering

Most of the score jump came from feature work, not from swapping one model for another. The raw columns were already useful, but they hid the borrower story. We got much better results once we started writing features that sounded like lending questions instead of spreadsheet formulas.

Feature family	Examples	Why it stayed in the final public pipeline
Affordability ratios	loan share of income, monthly payment share, asset coverage	These features turned raw balances into simple stress indicators and helped both targets.
Delinquency features	weighted late-payment counts, severity over history length, utilization interactions	They were consistently strong for `RiskTier` and also improved rate prediction.
Derogatory burden	charge-off, collection, bankruptcy, and public-record aggregates	A compact way to summarize past damage without relying on a single raw field.
Semantic missingness	renter-without-property, owner-without-mortgage, no-collateral flags	These features captured why a field was blank, which was often more useful than the filled value itself.
Controlled interactions	selected pairwise products between stress and balance features	A small interaction set helped the tree models find sharper non-linear boundaries.

Table 1: Feature groups that mattered enough to survive into the best public lane.

Figure 3 shows the same pattern from another angle. The two targets overlap, but they do not rank features in exactly the same way. Delinquency and utilization features dominate the classification side, while affordability and balance-sheet coverage matter a bit more for the rate regression.

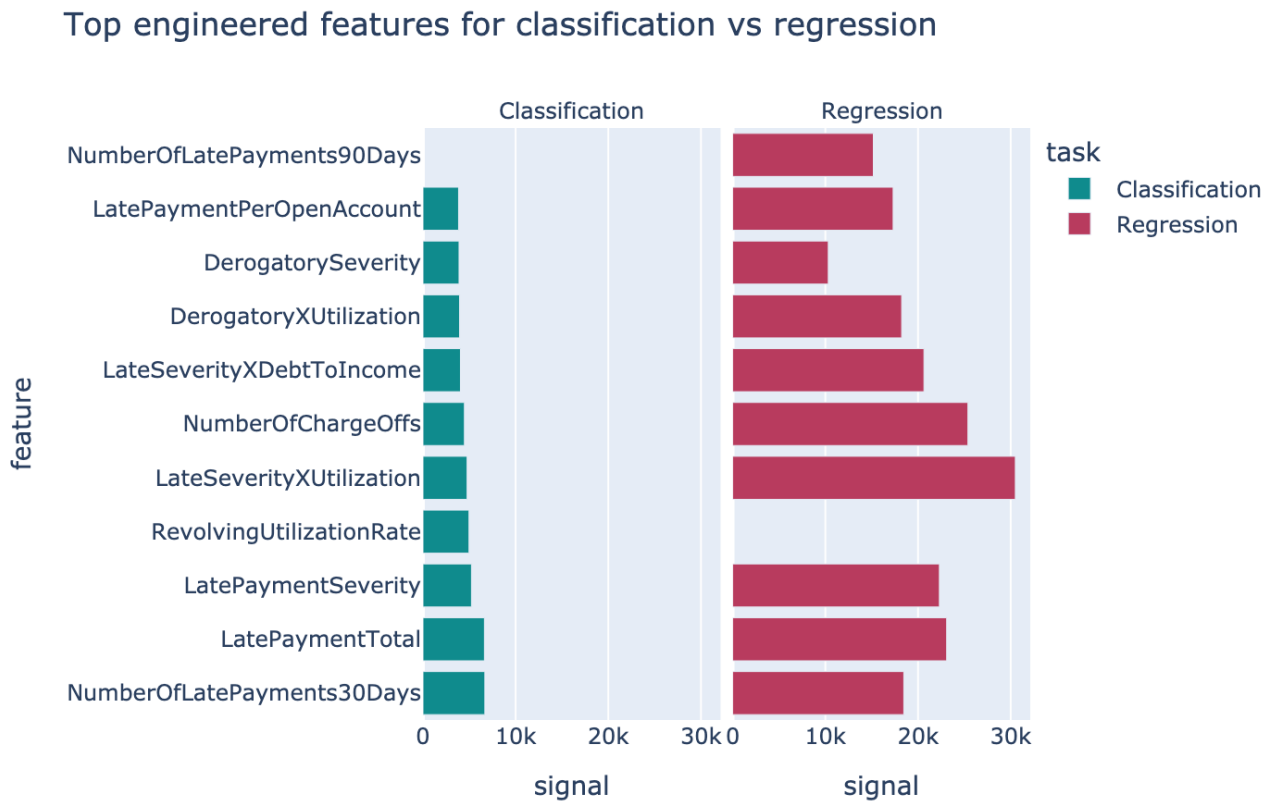


Figure 3: The strongest engineered features overlap across targets, but the ordering is not identical.

We also tracked how the score changed when major feature ideas were added. The important pattern is not that every extra trick helped, but that the first few feature families changed the game while later additions gave smaller and more selective gains. The overall project progression later in the report follows the same logic.

3 Model Selection & Final Pipeline

After the feature set settled down, the model story became much cleaner. Single models were good enough to guide the search, but the best public path came from letting several branch families vote in different ways:

1. XGBoost on the engineered numeric matrix.
2. LightGBM on the same processed frame.
3. CatBoost on the mixed feature table with categorical columns preserved.
4. ExtraTrees as a strong but simpler diversity branch.
5. An MPS neural branch to add a different error profile.

Instead of freezing mysterious blend weights inside the report code, the public repo now reads the original tuned branch configurations from saved overnight artifacts and fits the blend/meta combiner openly in code. That is closer to how the actual project worked and much easier to explain.

Model	Accuracy	R^2	Combined
`meta_overnight`	0.8477	0.8513	**0.8495**
`blend_overnight`	0.8424	0.8485	0.8455
`target_stats_blend`	0.8563	0.8552	0.8557
`mlp_meta_stack_v1`	0.8587	0.8556	0.8572
`mlp_weighted_blend_v1`	0.8600	0.8553	**0.8576**

Table 2: Representative validation checkpoints from the project.

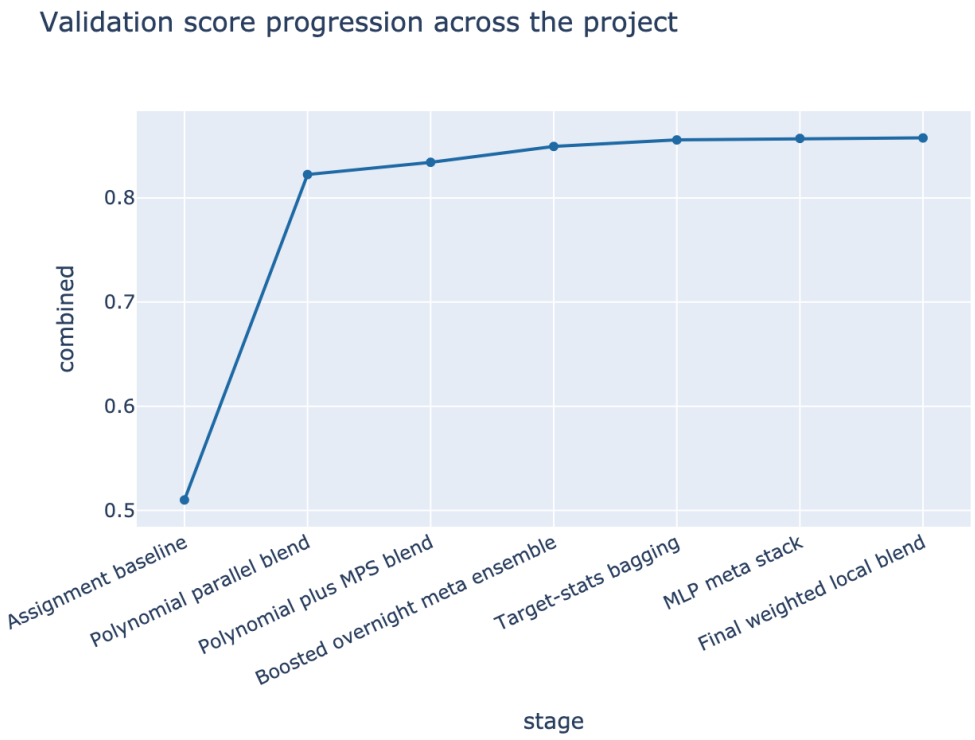


Figure 4: The project kept improving after the public-best lane, but those later gains came from heavier local-only combinations rather than a cleaner public submission path.

The final choice was mostly a trade-off between clarity and squeezing out the last few thousandths. We did reach a stronger local score later, but that route was harder to present cleanly and harder to justify as the main public notebook. For the repo and the report, `meta_overnight` is the right landing point: strong score, readable branch structure, and a direct path from raw CSVs to a final submission file.